

Grundlagen der Programmierung:

5. Java Repetitorium

Tutoren:

Daniel Grillmeyer, Ludwig Friedl, Jan Kerber, Jennifer Muck

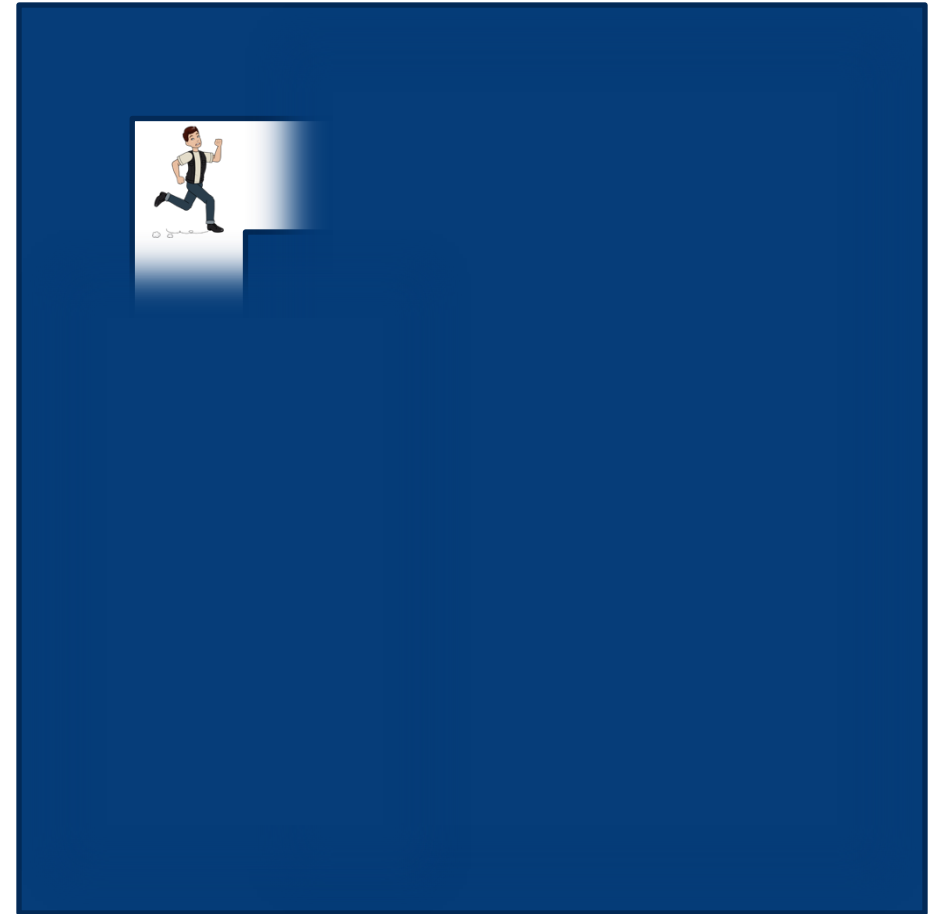
1. Lösung Programmieraufgabe 1
2. Vorstellung Programmieraufgabe 2
3. Puzzler 1: Foreach-Schleifen
4. Puzzler 2: Methoden
5. Puzzler 3: Methodenüberladung
6. Übung: Matrixkopie

LÖSUNG PROGRAMMIERAUFGABE 1

PROGRAMMIERAUFGABE 2

Programmieraufgabe 2

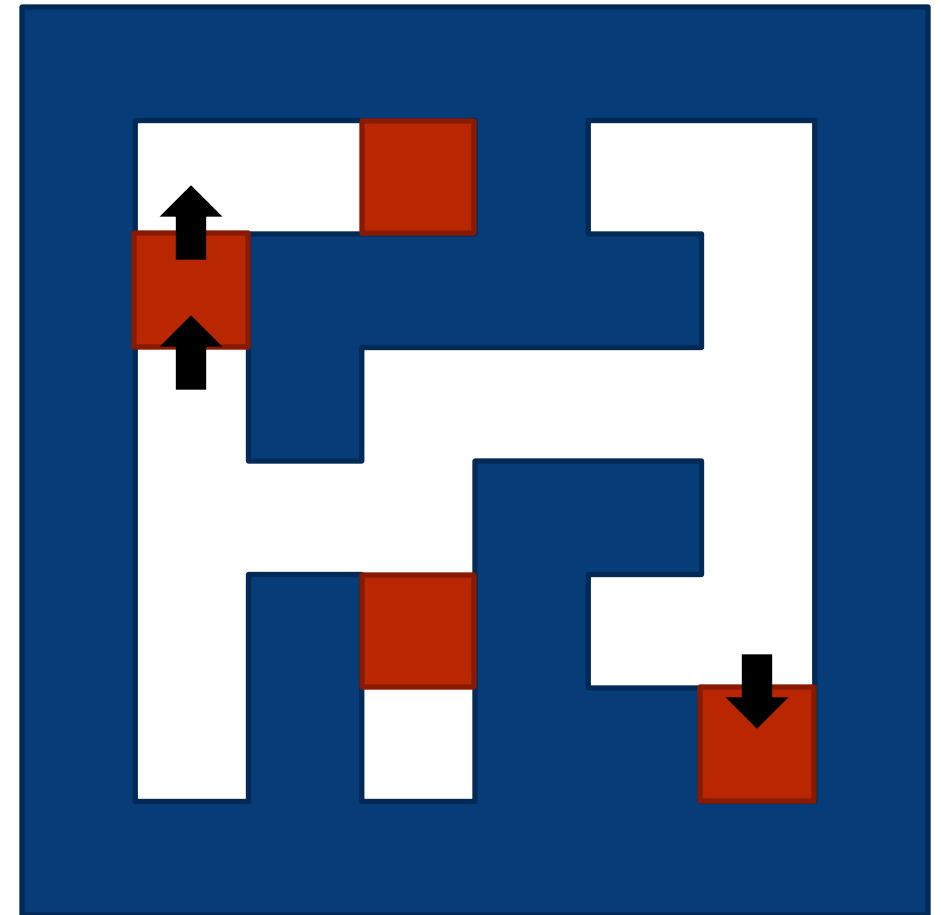
- Agent soll ein unbekanntes Labyrinth durchlaufen
- Währenddessen soll eine Karte erstellt werden
- Lösungsansatz: Trémaux Algorithmus



Programmieraufgabe 2

Weg: Nur ein möglicher Weg (auch Sackgasse)

► Weitergehen



Programmieraufgabe 2

Weg: Nur ein möglicher Weg (auch Sackgasse)

▶ Weitergehen

Platz: Fallunterscheidung



Programmieraufgabe 2

Weg: Nur ein möglicher Weg (auch Sackgasse)

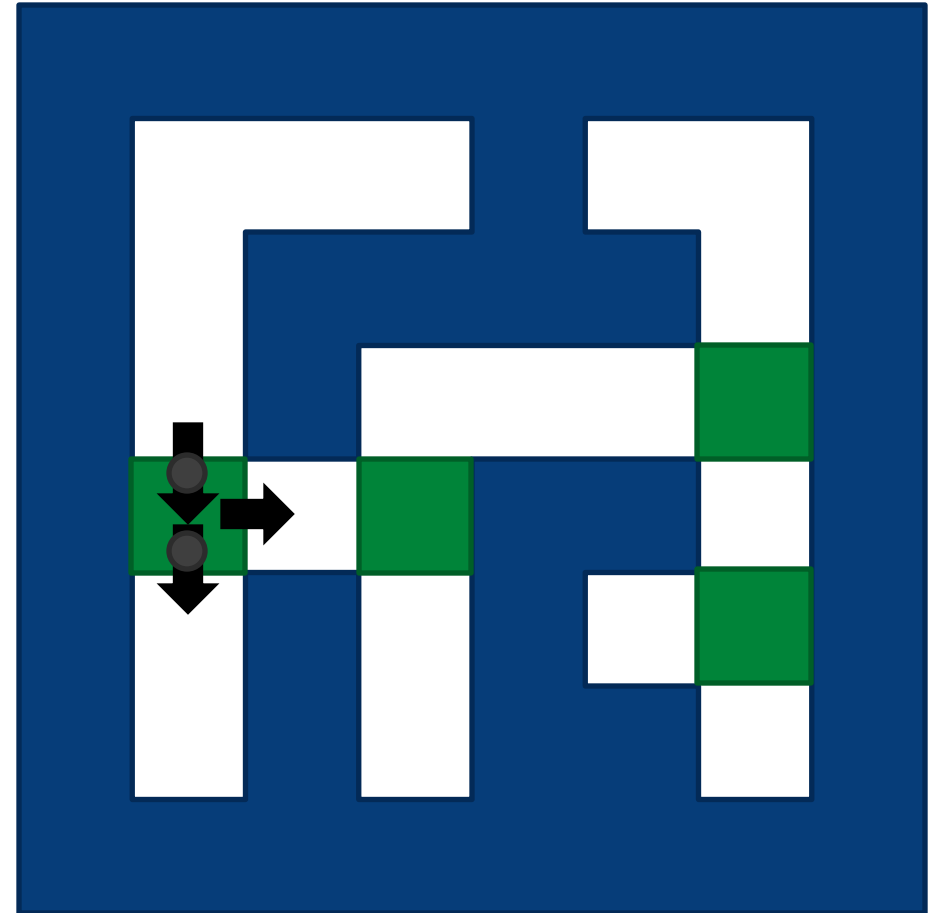
► Weitergehen

Platz: Fallunterscheidung

1. Es gibt keine Markierungen

► In beliebige Richtung weiter

● Markierungen setzen (in Quell- und Zielrichtung)



Programmieraufgabe 2

Weg: Nur ein möglicher Weg (auch Sackgasse)

▶ Weitergehen

Platz: Fallunterscheidung

1. Es gibt keine Markierungen

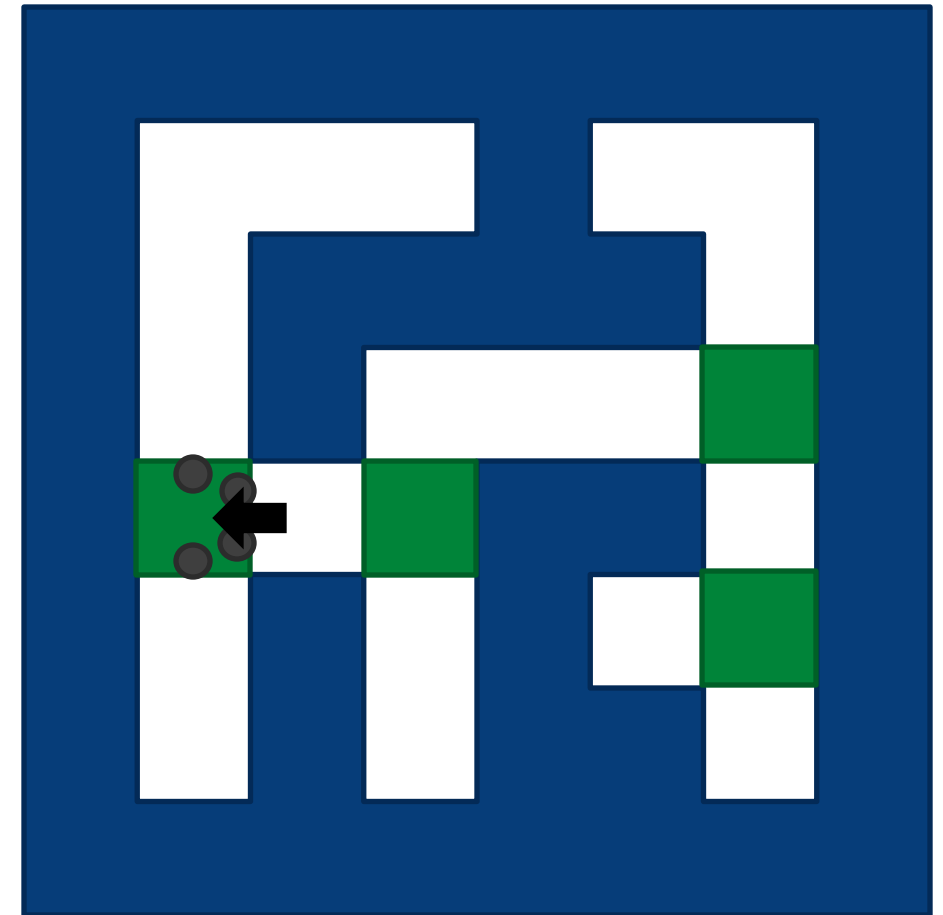
▶ In beliebige Richtung weiter

● Markierungen setzen (in Quell- und Zielrichtung)

2. Es gibt Markierungen und Quellrichtung ist nicht markiert

▶ Umkehren

● Markierungen setzen



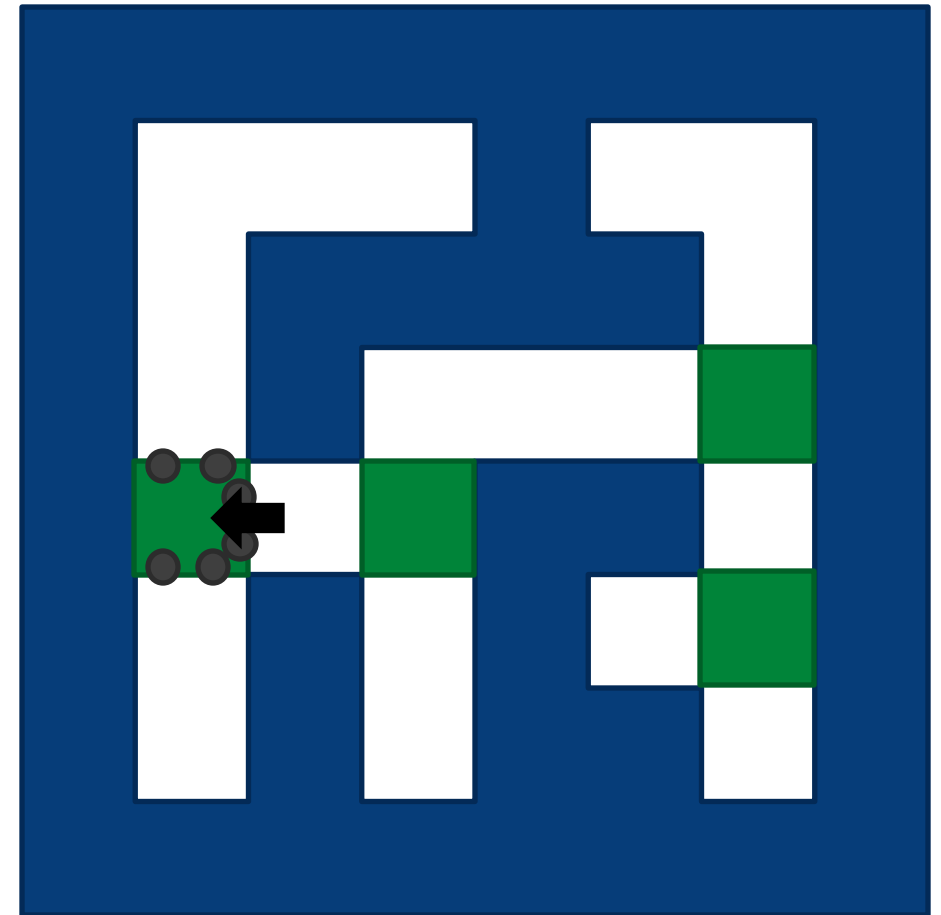
Programmieraufgabe 2

Weg: Nur ein möglicher Weg (auch Sackgasse)

► Weitergehen

Platz: Fallunterscheidung

1. Es gibt keine Markierungen
 - ▶ In beliebige Richtung weiter
 - Markierungen setzen (in Quell- und Zielrichtung)
2. Es gibt Markierungen und Quellrichtung ist nicht markiert
 - ▶ Umkehren
 - Markierungen setzen
3. Alle Richtungen haben mehr als eine Markierung
 - Fertig 



Programmieraufgabe 2

Weg: Nur ein möglicher Weg (auch Sackgasse)

▶ Weitergehen

Platz: Fallunterscheidung

1. Es gibt keine Markierungen

▶ In beliebige Richtung weiter

● Markierungen setzen (in Quell- und Zielrichtung)

2. Es gibt Markierungen und Quellrichtung ist nicht markiert

▶ Umkehren

● Markierungen setzen

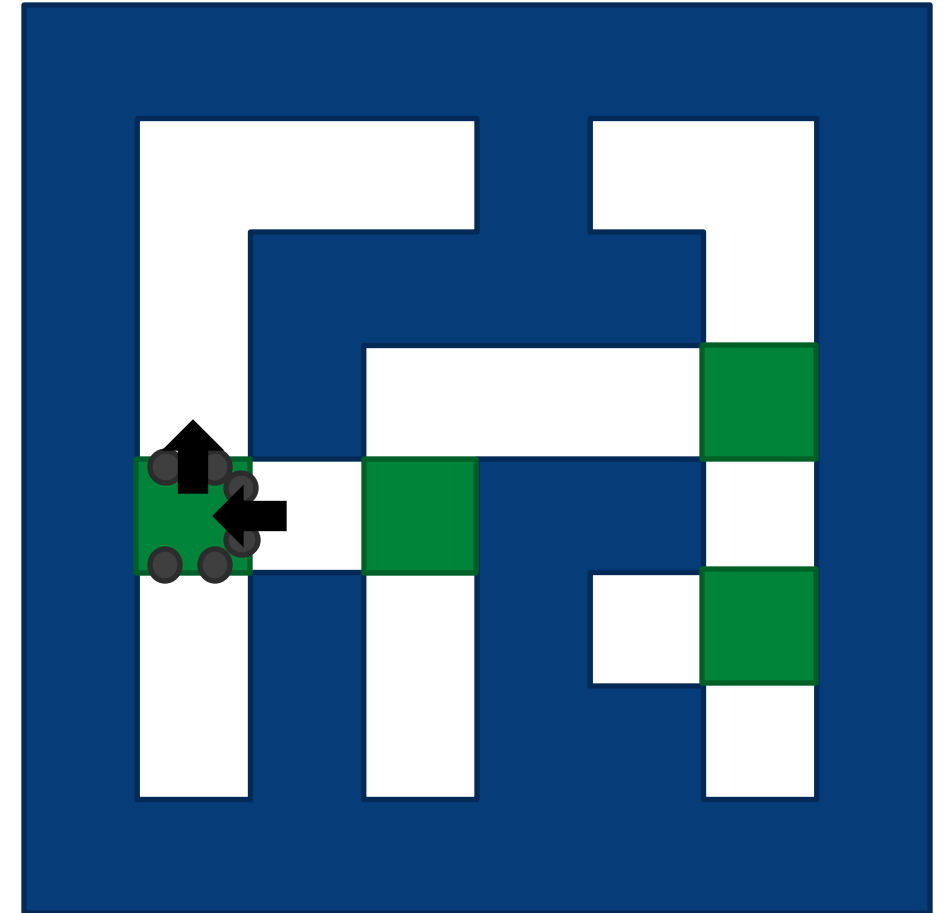
3. Alle Richtungen haben mehr als eine Markierung

→ Fertig |

4. Sonst

▶ Gehe in Richtung mit wenigsten Markierungen, aber nicht zurück!

● Markierungen setzen



Sie dürfen annehmen, dass es mind. einen Platz gibt!

Programmieraufgabe 2

- Was Sie können sollten:
 - Mehrdimensionale Arrays
 - Schleifen
- Eine main-Methode ist nicht gefordert, aber zu Testzwecken empfohlen.
- Nutzen Sie zum Testen auch die Beispiele in der Klasse Examples
- Sie können eigene Labyrinth mit Hilfe des MazeBuilder erstellen
- Hilfsmethoden sind erlaubt und erwünscht, zum Beispiel...
 - `private boolean isPath(Maze maze)`
Gibt false zurück, falls man sich auf einem Platz befindet, sonst true.
 - `private int orientation(int orientation, int direction)`
Gibt basierend auf der aktuellen Orientierung und Bewegungsrichtung die neue Orientierung zurück.

Beispiellabyrinth

```
#####h#
#                #
# #####
# # #          #
# # # ##### #
# #   #   # #
# #### # # # #
#   # # # # #
### # # # # #
#   #       # #
### ##### #
#                #
#####
```

Puzzler 1: Foreach-Schleifen

```
1 int[] values = {4, 5, 6};  
2 for (int x : values) {  
3     x = 1;  
4 }
```

Welche Werte sind im Array values enthalten?

{4, 5, 6}

{1, 1, 1}

Etwas anderes

Puzzler 1: Foreach-Schleifen

```
1 int[] values = {4, 5, 6};  
2 for (int x : values) {  
3     x = 1;  
4 }
```

Welche Werte sind im Array values enthalten?

{4, 5, 6}

{1, 1, 1}

Etwas anderes

Warum?

- Die Iterationsvariable x ist nur eine Kopie des Array-Elements

Puzzler 2: Methoden

Gegeben ist die Klasse **Puzzler** mit einigen implementierten Methoden.
Der genaue Inhalt der Methoden ist nicht relevant.

```
1 public class Puzzler {  
2     public static long methodA(long a, long b) { ... }  
3  
4     public static String methodB(String a, int b) { ... }  
5 }
```

Welche der folgenden Methoden können zu der Klasse **Puzzler** hinzugefügt werden
ohne einen Compiler-Fehler zu erhalten?

6	<code>public static long methodA(int a, long b) { ... }</code>	Funktioniert
7	<code>public static long methodA(long a, long b, long c) { ... }</code>	Funktioniert
8	<code>public static long methodA(long b, long a) { ... }</code>	Fehler
9	<code>public static long methodB(long a, long b) { ... }</code>	Funktioniert
10	<code>public static String methodB(int b, String a) { ... }</code>	Funktioniert
11	<code>public static long methodB(String a, int b) { ... }</code>	Fehler

Puzzler 2: Erklärung

- Die Signatur einer Methode muss einmalig sein innerhalb einer Klasse
- Zur Signatur einer Methode gehören
 - Name der Methode
 - Anzahl der Parameter
 - Art der Parameter
 - Reihenfolge der Parametertypen
- Zur Signatur gehören insbesondere **NICHT**
 - ~~Rückgabedatentyp~~
 - ~~Zugriffs-Modifizierer~~
 - ~~Schlüsselwort "static"~~

Puzzler 3: Methodenüberladung

Gegeben ist die Klasse **Puzzler** mit einigen implementierten Methoden.
Der genaue Inhalt der Methoden ist nicht relevant.

```
1 public class Puzzler {  
2     public static void methodA(long a, long b) { ... }  
3     public static void methodA(int a, long b) { ... }  
4     public static void methodA(int a, int b) { ... }  
5     public static void methodA(short a, short b) { ... }  
6 }
```

A
B
C
D

Entscheiden Sie, welche Implementierung von **methodA** in den jeweiligen Beispielen aufgerufen wird

```
7 long l = Long.MAX_VALUE;  
8 byte b = Byte.MAX_VALUE;  
9 int i = 0;
```

```
10 methodA(1, 1);  
11 methodA(1, i);  
12 methodA(b, i);  
13 methodA((byte) 1, i);  
14 methodA(3, 2);  
15 methodA(3.0, i);
```

A
A
C
C
C
Fehler

Puzzler 3: Erklärung

- Seien im Folgenden Methoden (Methodenaufrufe und Methodendeklarationen) über ihren Methodennamen und Parametertypen identifiziert
- Beispiel: `public static void calculate(int myInt, String myString){...}`
wird zu: `calculate(int, String)`

Puzzler 3: Erklärung

- Sei $m(P_1, P_2, \dots, P_n)$ ein Methodenaufruf, dann ist eine Methodendeklaration $m'(S_1, S_2, \dots, S_o)$ **aufrufbar** $:\Leftrightarrow$ alle folgenden Punkte treffen zu:
 - Die Methode m' ist zugreifbar (abhängig von Modifikatoren, z.B. `public`)
 - Der Methodename stimmt überein (also $m = m'$)
 - Die Parameteranzahl stimmt überein (also $n = o$)
 - Für alle i mit $1 \leq i \leq n$ gilt:
 P_i ist in S_i durch elementare Typvergrößerungen oder Vergrößerungen von Referenzdatentypen umwandelbar oder $P_i = S_i$

Puzzler 3: Erklärung

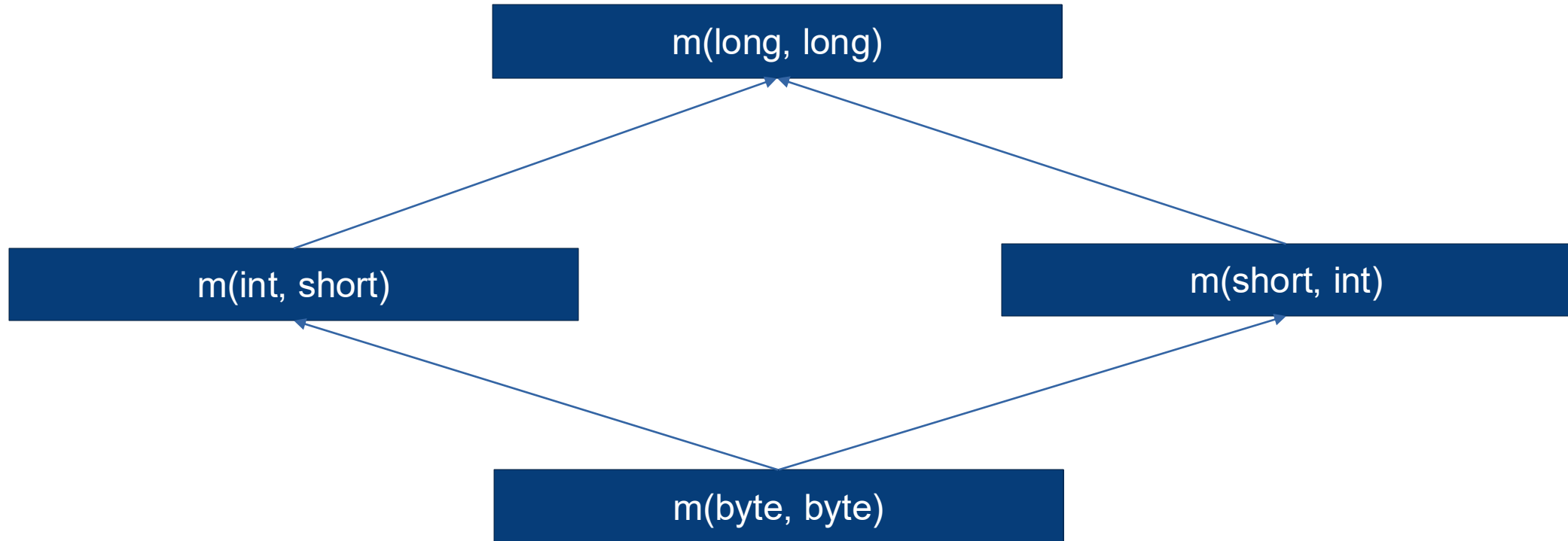
- Eine Methodendeklaration $m(S_1, S_2, \dots, S_n)$ heißt **spezifischer** als $m(T_1, T_2, \dots, T_n) : \Leftrightarrow$
- Für alle i mit $1 \leq i \leq n$ gilt:
 S_i ist in T_i durch elementare Typvergrößerungen oder Vergrößerungen von Referenzdatentypen umwandelbar oder $T_i = S_i$
 - Für mindestens ein i mit $1 \leq i \leq n$ gilt: $T_i \neq S_i$

Die Signaturen
dürfen nicht
gleich sein!

Puzzler 3: Erklärung

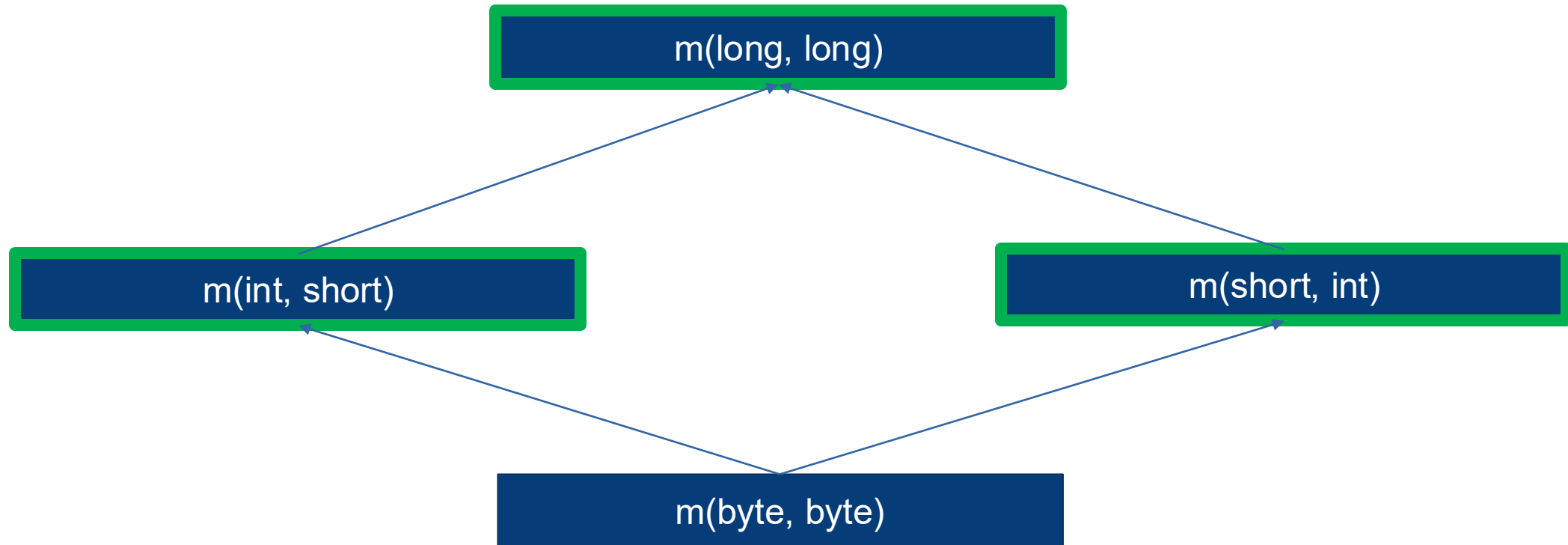
- Eine Methodendeklaration ist für einen Methodenaufruf **maximal spezifisch**, falls sie aufrufbar ist und keine andere aufrufbare Methode spezifischer als sie ist
- Existiert unter den aufrufbaren Methoden genau eine maximal spezifische Methode, so ist sie die **spezifischste** und wird aufgerufen
- Existieren **zwei** oder **mehr** maximal spezifische Methodendeklarationen, so heißt der Methodenaufruf **mehrdeutig** und verursacht eine entsprechende **Fehlermeldung**

Puzzler 3: Erklärung



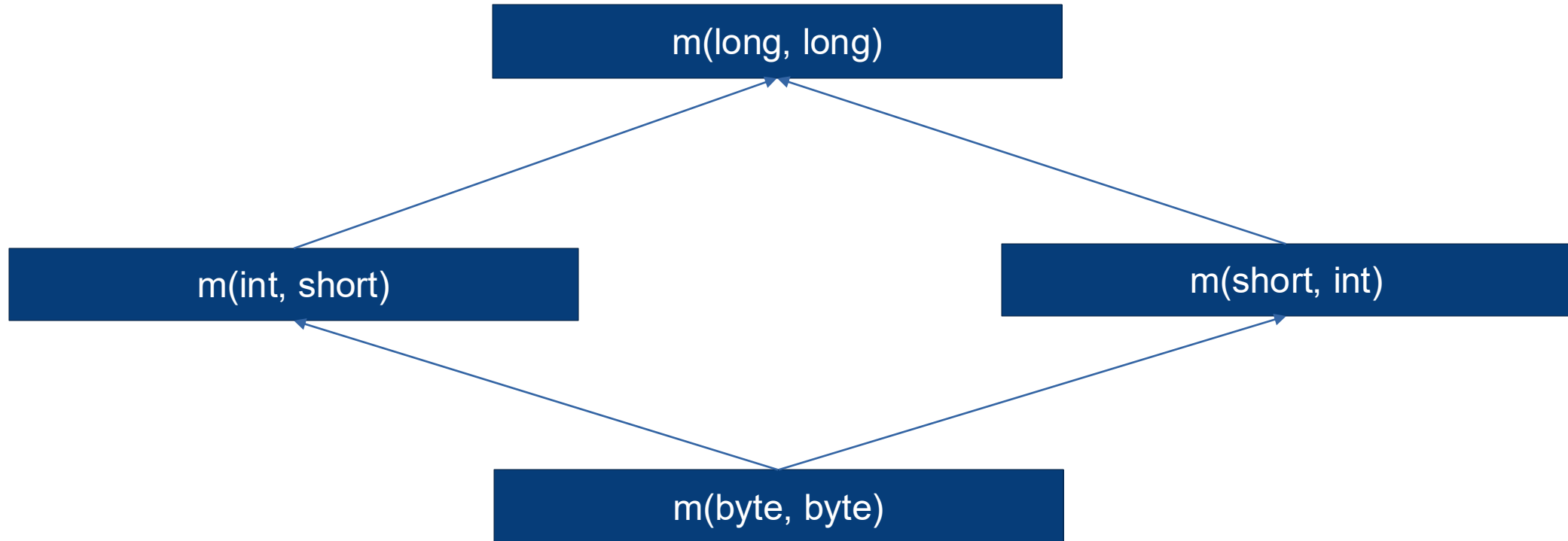
Welche Methoden sind **aufrufbar** für den Methodenaufruf `m(short, short)`?

Puzzler 3: Erklärung



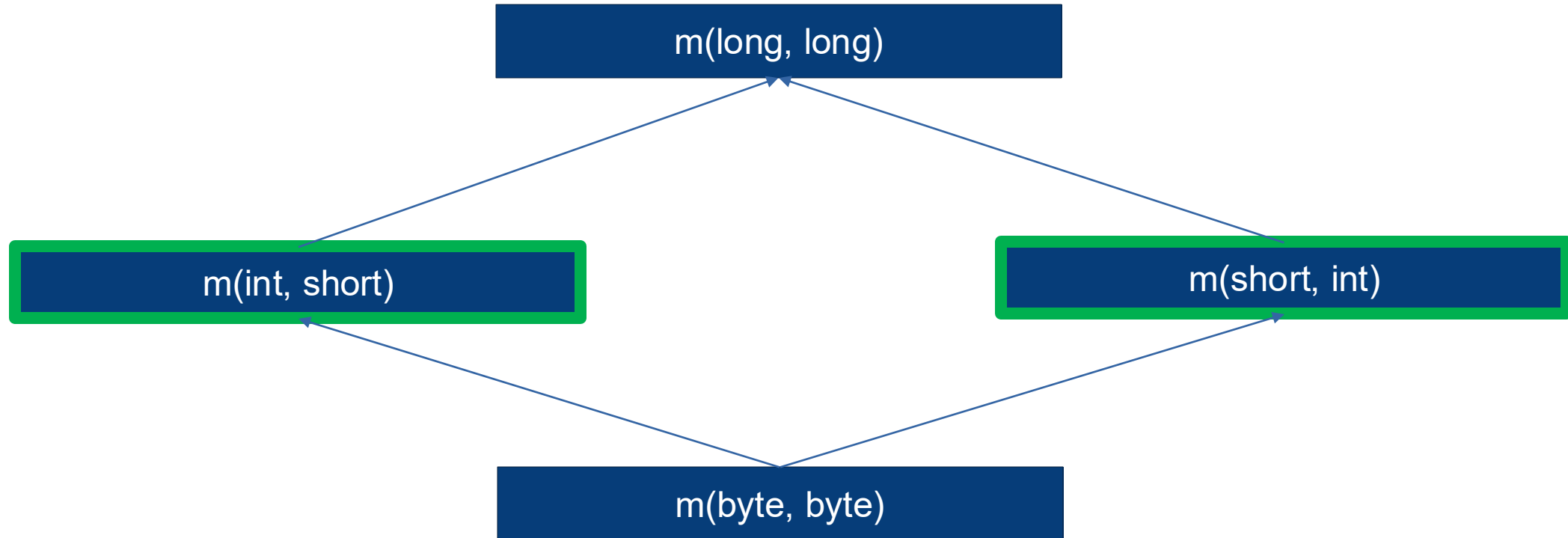
Welche Methoden sind **aufrufbar** für den Methodenaufruf `m(short, short)`?

Puzzler 3: Erklärung



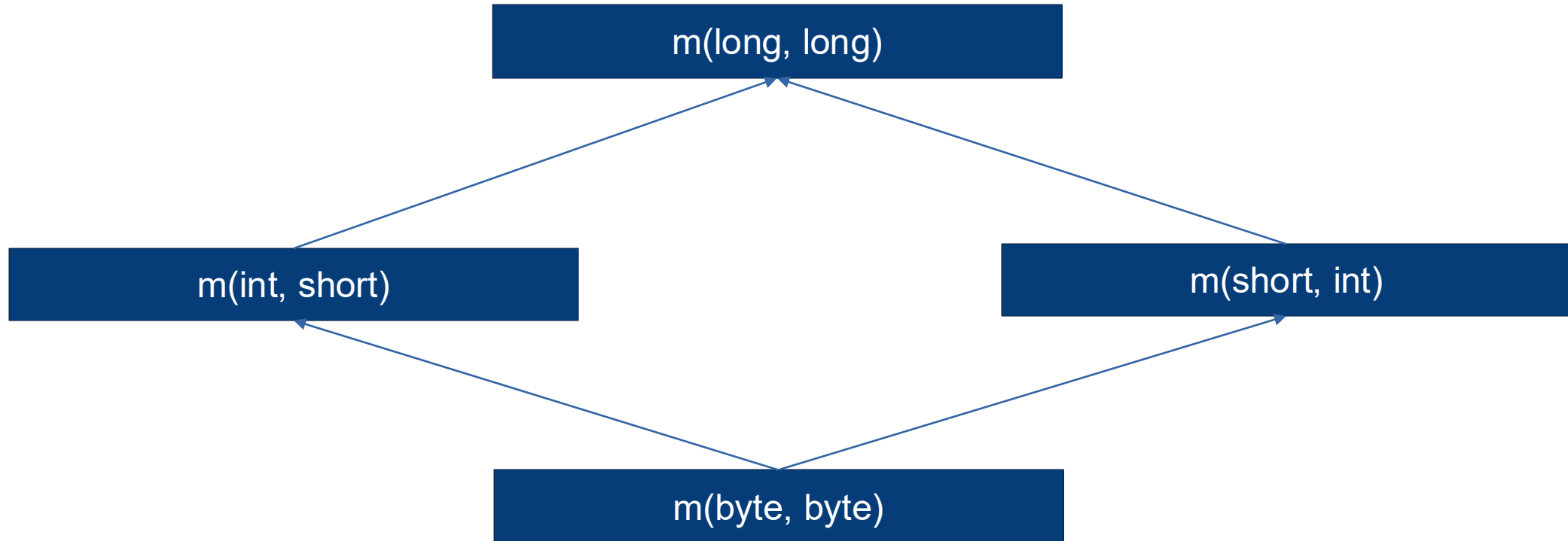
Welche Methoden sind **maximal spezifisch** für den Methodenaufruf `m(short, short)`?

Puzzler 3: Erklärung



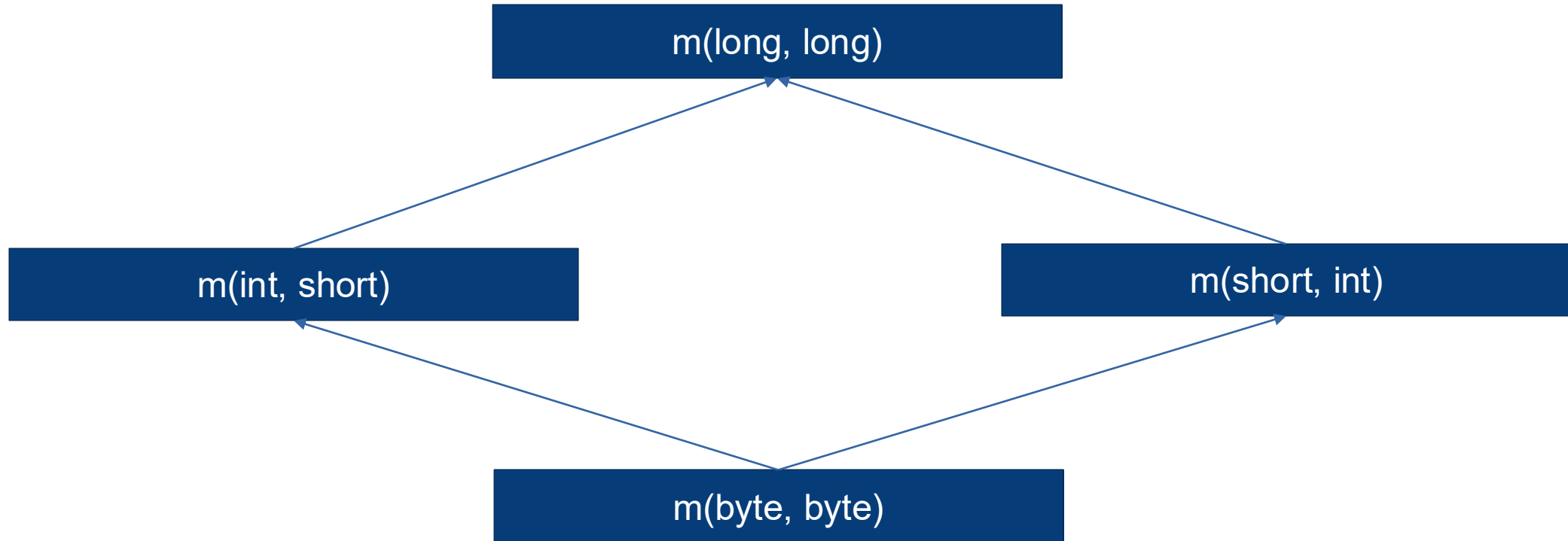
Welche Methoden sind **maximal spezifisch** für den Methodenaufruf `m(short, short)`?

Puzzler 3: Erklärung



Welche Methode ist **die spezifischste** für den Methodenaufruf `m(short, short)`?

Puzzler 3: Erklärung



Welche Methode ist **die spezifischste** für den Methodenaufruf `m(short, short)`? **Keine!**

Übung: Matrixkopie

Implementiere die Methode `copy`, die ein mehrdimensionales `int`-Array (d.h. eine Matrix) kopiert! Achte auf unterschiedliche Längen innerhalb der Matrix!

```
1 public class Puzzler {
2     public static void main(String[] args) {
3         int[][] matrix = {{1}, {2, 3}, {4, 5, 6}};
4         int[][] copied = copy(matrix);
5     }
6
7     public static int[][] copy(int[][] matrix) {
8
9
10
11
12
13
14
15
16 }
17 }
```

Übung: Matrixkopie

Implementiere die Methode `copy`, die ein mehrdimensionales `int`-Array (d.h. eine Matrix) kopiert! Achte auf unterschiedliche Längen innerhalb der Matrix!

```
1 public class Puzzler {
2     public static void main(String[] args) {
3         int[][] matrix = {{1}, {2, 3}, {4, 5, 6}};
4         int[][] copied = copy(matrix);
5     }
6
7     public static int[][] copy(int[][] matrix) {
8         int[][] result = new int[matrix.length][];
9         for (int i = 0; i < result.length; i++) {
10             result[i] = new int[matrix[i].length];
11             for (int j = 0; j < result[i].length; j++) {
12                 result[i][j] = matrix[i][j];
13             }
14         }
15         return result;
16     }
17 }
```